Python Cursus: Week 5

Leerdoelen

- ☐ Niet-lineaire fits uitvoeren met `scipy.optimize.curve_fit`.
- ☐ Fysische parameters uit een fit afronden en interpreteren.
- ☐ Onzekerheden bepalen uit de covariantiematrix.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Gebruik ChatGPT verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Bijeenkomst 5: Curve fitting met SciPy

Model

Het uitvoeren van een (niet-)lineaire regressie op een dataset vereist eerst een fysisch of empirisch model dat het verwachte gedrag van de data beschrijft. Dit model bevat één of meerdere parameters die niet direct bekend zijn en uit de data moeten worden bepaald. Het doel van curve fitting is om deze parameters zo te kiezen dat het verschil tussen het model en de gemeten data zo klein mogelijk wordt, meestal in de zin van een kleinste-kwadratencriterium. In Python kan dit proces efficiënt worden uitgevoerd met de functie `curve_fit` uit de `SciPy`-bibliotheek. In deze cursus gebruiken we als model een exponentiële afname, die bijvoorbeeld optreedt bij ontlaadprocessen of relaxatieverschijnselen.

$$N(t) = N_0 e^{-t/\tau},$$

Curve fitting

De functie `curve_fit` bepaalt de modelparameters door het verschil tussen het model en de gemeten data te minimaliseren volgens het gewogen kleinste-kwadratenprincipe. Concreet wordt de som van de kwadraten van de residuen geminimaliseerd (gecorrigeerd voor het aantal vrije datapunten):

$$\tilde{\chi}^2 = \frac{1}{N-d} \sum_i \frac{\left(y_i - f(t_i, \theta)\right)^2}{\sigma_i^2},$$

waar y_i de gemeten waarden zijn, $f(t_i, \theta)$ het model, θ de te bepalen parameters en σ_i de onzekerheid op het betreffende datapunt.

De belangrijkste invoer voor `curve_fit` is een model-functie $f(t, \theta)$ (in ons geval dus een exponentieel verband), een array met de onafhankelijke variabele (bijvoorbeeld tijd t) en een array met de gemeten waarden $N(t)$. Optioneel kunnen ook meetfouten, beginwaarden en grenzen voor de parameters worden meegegeven.

De uitvoer bestaat uit de beste schatting van de modelparameters en de covariantiematrix. Uit de covariantiematrix wordt de onzekerheden op deze parameters bepaald.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def decay(t, y0, tau):
    return y0 * np.exp(-t/tau)

# gesimuleerde data met ruis
t = np.linspace(0, 10, 200)
y = decay(t, 5.0, 2.0) + 0.2*np.random.randn(len(t))

popt, pcov = curve_fit(decay, t, y)
y0_fit, tau_fit = popt
y0_err, tau_err = np.sqrt(np.diag(pcov))

plt.plot(t, y, ".", label="data")
plt.plot(t, decay(t, *popt), "-", label="fit")
plt.xlabel("t (s)")
plt.ylabel("y(t)")
plt.legend()
plt.grid()
plt.show()
```

```
print("y0 =", y0_fit, "+/-", y0_err)
print("tau =", tau_fit, "+/-", tau_err)
```

Korte opdracht

Voer een fit uit voor een curve die jij krijgt of zelf genereert. Rapporteer de optimale parameters en hun onzekerheden. Rapporteer ook de fysische parameters.